



INSTITUTO POLITÉCNICO NACIONAL ESCUELA SUPERIOR DE CÓMPUTO

PRÁCTICA 7

REGISTRO DE DESPLAZAMIENTO 74HC595A

DISEÑO DE SISTEMAS DIGITALES

4CM4

INTEGRANTES:

MONTEALEGRE ROSALES DAVID URIEL
SALAUZ HERNANDEZ NERICK FRANCISCO
VÁZQUEZ BLANCAS CESAR SAID

PROFESOR:

AGUILAR SANCHEZ FERNANDO

INSTITUTO POLITÉCNICO NACIONAL



FECHA DE ENTREGA: 12 DE ENERO DE 2025

1. OBJETIVO

El alumno verificará el funcionamiento del CI 74HC595A, el cual es un registro de desplazamiento con entrada serial / salida serial-paralelo con carga de 8 bits.

2. MATERIAL Y EQUIPO EMPLEADO

- 2 CI 74HC595A
- 8 resistencias $330\ \Omega$ a $1/4\ W$
- 8 resistencias $1\ K\Omega$ a $1/4\ W$
- 2 Displays Cátodo o Ánodo común
- Dip switch de 8

3. INTRODUCCIÓN TEÓRICA:

En el campo del diseño de sistemas digitales, los **registros de desplazamiento** son componentes fundamentales que permiten la manipulación eficiente de datos en sistemas electrónicos. Estos dispositivos son ampliamente utilizados en aplicaciones donde se requiere la conversión de datos seriales a paralelos, el almacenamiento temporal de información, o la reducción del número de pines de control necesarios para manejar múltiples salidas.

El **74HC595A** es un circuito integrado que representa un ejemplo típico de un registro de desplazamiento con salida serial y paralela. Este dispositivo cuenta con un registro de 8 bits y un latch (o almacenamiento temporal) que mantiene los datos en las salidas paralelas hasta que se actualicen con nuevos valores. Su diseño lo hace ideal para aplicaciones que involucren microcontroladores o sistemas digitales con recursos limitados.

Características Principales del 74HC595A

1. Entrada Serial y Salida Paralela:

- El 74HC595A permite cargar datos en serie, bit a bit, utilizando una sola línea de datos junto con una señal de reloj. Estos datos pueden ser transferidos posteriormente a las salidas paralelas, optimizando el uso de pines del sistema controlador.

2. Latch de Salida:

- Las salidas paralelas están asociadas a un latch interno que almacena los datos. Esto garantiza que los valores de salida permanezcan estables, incluso si el registro de desplazamiento recibe nuevos datos. La estabilidad en las salidas es crucial en sistemas donde las señales controlan dispositivos sensibles como LEDs, displays o relés.

3. Capacidad de Cascada:

- Una de las características más destacadas del 74HC595A es su capacidad para ser encadenado con otros registros de desplazamiento. Esto se logra utilizando

su salida serial para alimentar la entrada serial del siguiente dispositivo en la cadena, permitiendo la expansión del número de salidas paralelas disponibles.

4. Control Sencillo:

- El circuito requiere tres señales principales para su operación:
 - **SHCP (Shift Clock):** Avanza el registro de desplazamiento, permitiendo que los datos ingresen o se desplacen.
 - **STCP (Storage Clock):** Transfiere los datos desde el registro de desplazamiento al latch de salida.
 - **OE (Output Enable):** Habilita o deshabilita físicamente las salidas paralelas, lo que puede ser útil para reducir interferencias durante el proceso de carga de datos.

5. Compatibilidad y Velocidad:

- El 74HC595A pertenece a la familia de circuitos CMOS de alta velocidad, lo que le permite operar en un rango amplio de frecuencias, haciéndolo compatible con sistemas que requieren respuesta rápida.

Importancia en el Diseño de Sistemas Digitales

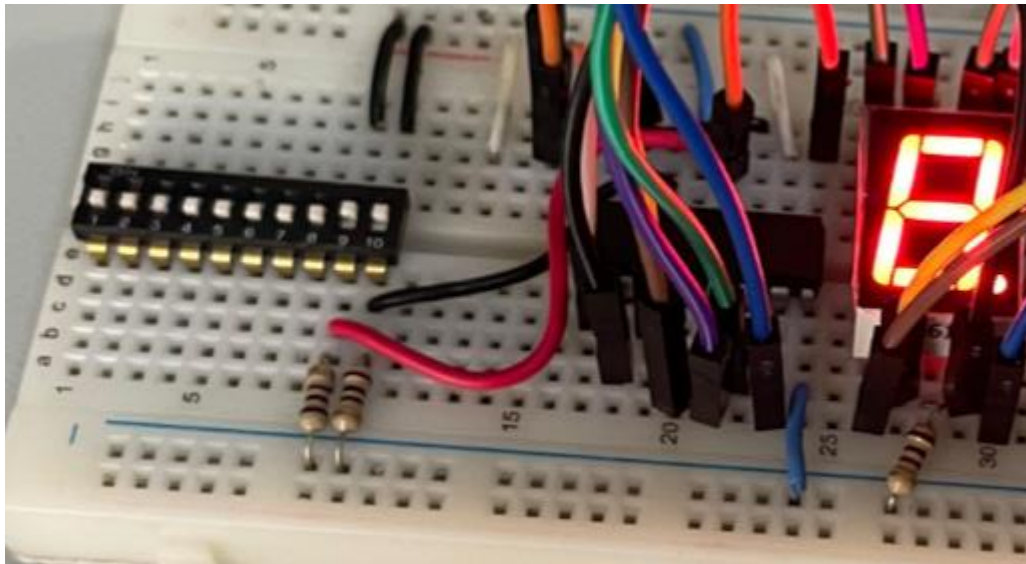
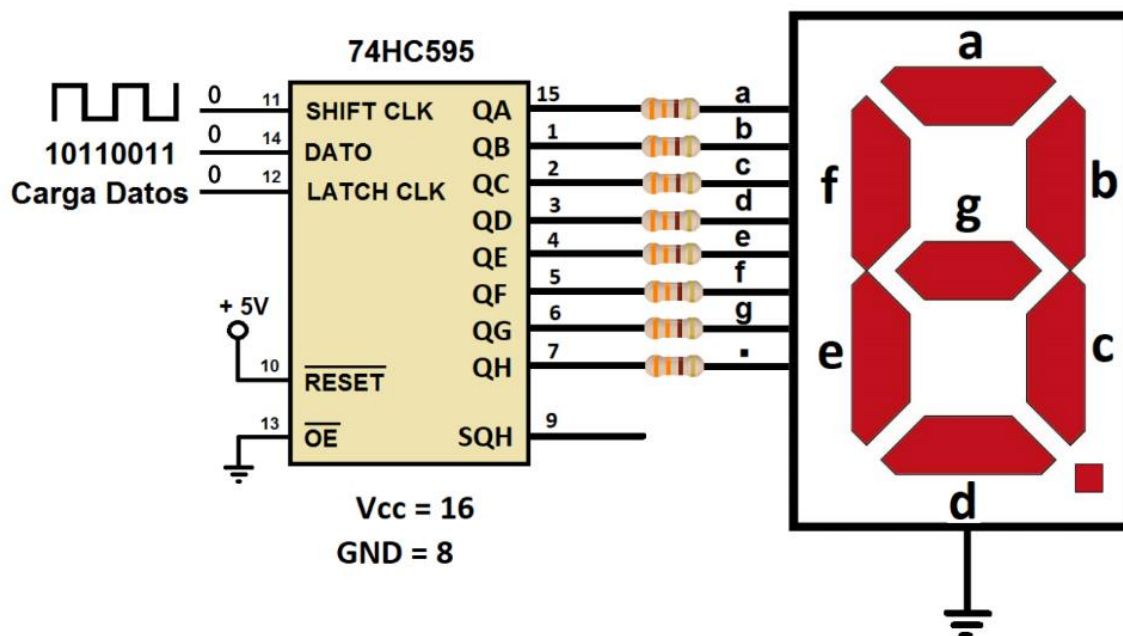
El 74HC595A es una solución eficiente para situaciones donde el sistema controlador, como un microcontrolador o FPGA, tiene un número limitado de pines disponibles pero necesita manejar múltiples dispositivos. Por ejemplo, es común encontrar este registro de desplazamiento en aplicaciones de control de matrices de LEDs, displays de 7 segmentos, y sistemas de iluminación decorativa. Su diseño modular permite escalar la cantidad de salidas simplemente añadiendo más registros en cascada.

Además, al operar con una interfaz serial, este dispositivo reduce significativamente el ruido eléctrico en comparación con una transmisión completamente paralela, lo que es beneficioso en entornos donde la interferencia puede ser un problema. Su latch integrado asegura la estabilidad de las salidas, minimizando errores en sistemas sensibles.

En resumen, el **74HC595A** es un componente versátil y esencial para optimizar recursos en el diseño de sistemas digitales, facilitando la integración de múltiples dispositivos y garantizando un control eficiente y confiable. Su estudio y comprensión son fundamentales para desarrollar habilidades en el manejo de señales digitales y la implementación de soluciones innovadoras en proyectos electrónicos.

DESARROLLO

Actividad 1. Armar el siguiente circuito para visualizar los números del 0 a 9 es el de la figura 3.

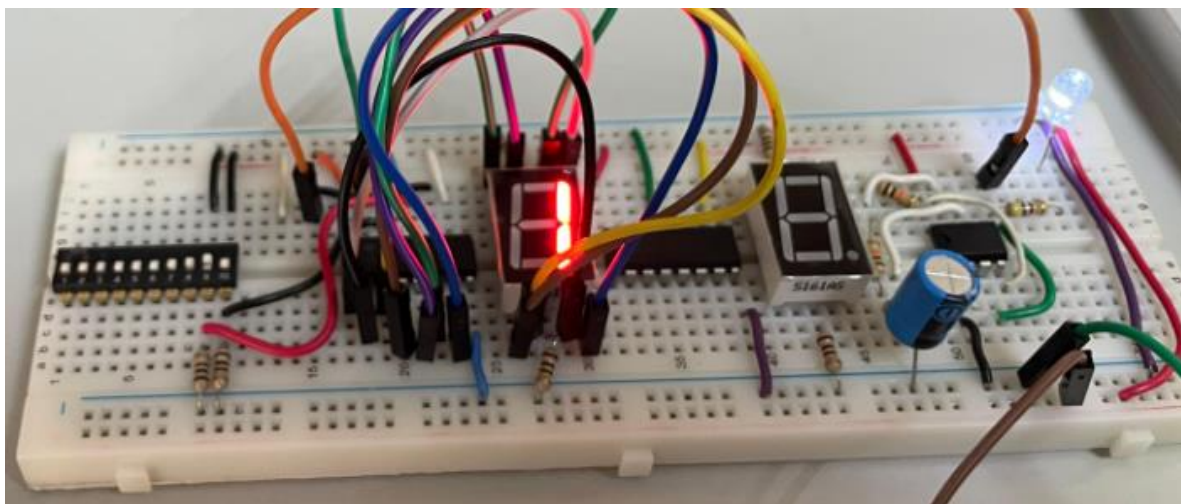
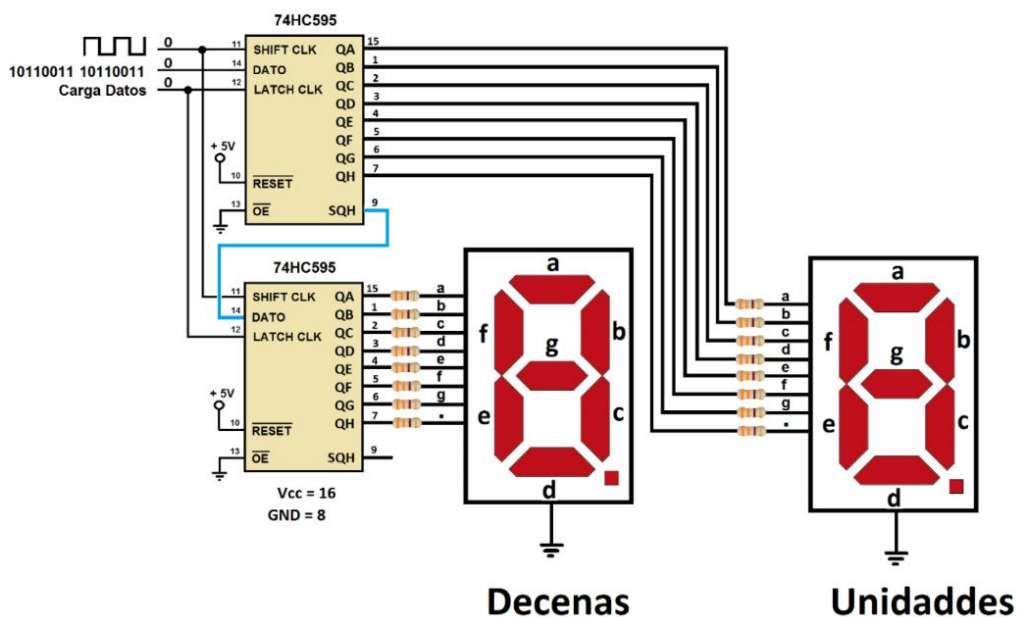


Actividad 2. Armar el circuito de la figura 5 para visualizar los números del 00 a 99.

Recuerde que el valor se ingresa por la terminal DATO, la información ahora es de 16 bit, primero ingresamos el valor de la Decena y después de la Unidad, es decir, si quiero ver en el Display el valor 73, primero ingresamos el valor del siete y después el tres. 0 0 0 0 0 1 1 1 0 1 0 0 1 1 1 1

7 3

No olvidemos que los 16 bits ingresan por un solo CI. Al estar conectados en cascada (línea azul de la figura 5), la información del valor se transfiere al siguiente Integrado.



Actividad 3. Diseña junto con la GAL22V10 y el circuito de la figura 5, la automatización del circuito para generar un contador del 00 al 99.

Gal 1:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity cont4 is
    port (
        clk: in std_logic;
        Q: out std_logic_vector(3 downto 0) );
end cont4;
architecture arqcont of cont4 is
    signal count: unsigned(3 downto 0) := (others => '0');
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if count = "1001" then -- Reinicia a 0 después de contar hasta 9
                count <= "0000";
            else
                count <= count + 1;
            end if;
        end if;
    end process;
    Q <= std_logic_vector(count);
end arqcont;
```

Gal 2:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity unidades is
    Port (x : in std_logic_vector(3 DOWNTO 0); clk : in std_logic; latch_clk : out std_logic;
          data_out : out std_logic );
end unidades;
architecture arq_unidades of unidades is
    signal z : std_logic_vector(7 downto 0);
    signal bit_index : integer range 0 to 7 := 0;
begin
    convertidor: PROCESS (x)
    BEGIN
        CASE x IS
            WHEN "0000" => z <= "11111101"; -- 0 10111111
            WHEN "0001" => z <= "01100001"; -- 1 10000110
            WHEN "0010" => z <= "11011011"; -- 2 11011011
            WHEN "0011" => z <= "11110011"; -- 3 11001111
            WHEN "0100" => z <= "01100111"; -- 4 11100110
            WHEN "0101" => z <= "10110111"; -- 5 11101101
            WHEN "0110" => z <= "10111111"; -- 6 11111101
            WHEN "0111" => z <= "11100001"; -- 7 10000111
            WHEN "1000" => z <= "11111111"; -- 8
            WHEN "1001" => z <= "11110111"; -- 9 11101111
        END CASE;
    END PROCESS;
```

```

PROCESS (clk)

    BEGIN

        IF rising_edge(clk) THEN

            latch_clk<='1';

            data_out <= z(bit_index);

            IF (bit_index=7) THEN

                latch_clk<='0';

                bit_index<=0;

            ELSE

                latch_clk<='1';

                bit_index <= bit_index+1;

            END IF;

        END IF;

    END PROCESS;

end arq_unidades;

```

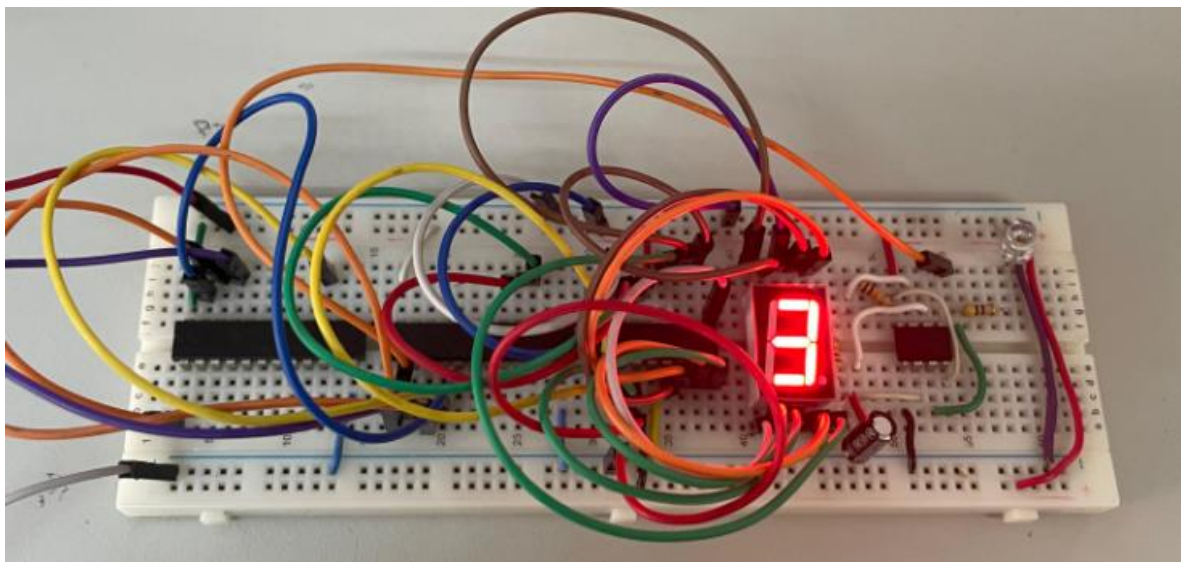
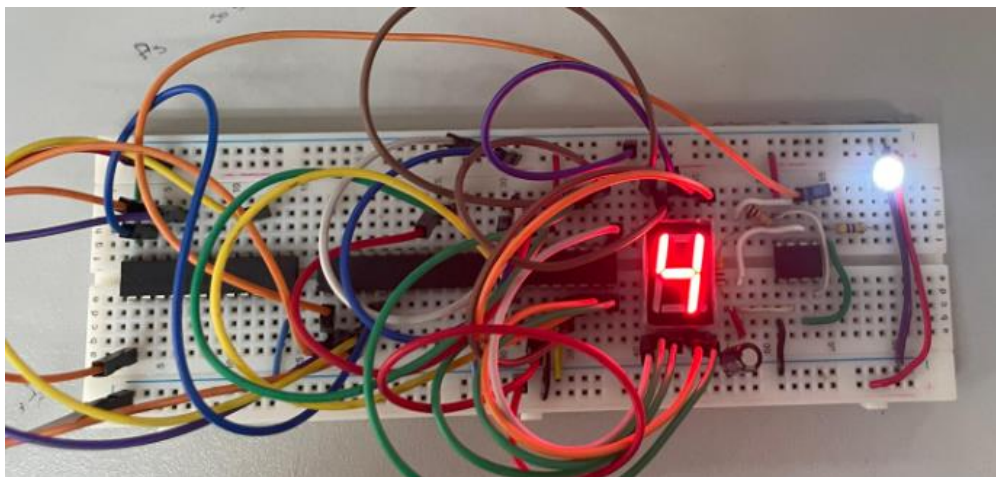
Gal 1 pines:

C22V10		
clk =	1	24 * not used
reset =	2	23 = q(2)
load =	3	22 = q(0)
enp =	4	21 * not used
not used *	5	20 * not used
not used *	6	19 * not used
not used *	7	18 * not used
not used *	8	17 * not used
not used *	9	16 * not used
not used *	10	15 = q(1)
not used *	11	14 = q(3)
not used *	12	13 * not used

Gal 2 pines:

```
C22V10

clk =| 1|
x(3) =| 2|
x(2) =| 3|
x(1) =| 4|
x(0) =| 5|
not used *| 6|
not used *| 7|
not used *| 8|
not used *| 9|
not used *|10|
not used *|11|
not used *|12|
|24| * not used
|23|= (bit_index_I..
|22|= (bit_index_I..
|21| * not used
|20| * not used
|19| * not used
|18|= data_out
|17| * not used
|16| * not used
|15|= latch_clk
|14|= (bit_index_I..
|13| * not used
```



OBSERVACIONES Y CONCLUSIONES

Montealegre Rosales David Uriel

La implementación del registro de desplazamiento 74HC595, combinada con la automatización mediante la GAL22V10 y el lenguaje VHDL, permitió diseñar un sistema eficiente para realizar el conteo de 0 a 9 en un display de 7 segmentos. En esta práctica, comprendí cómo los pulsos del reloj (SHIFT_CLK) y las señales de latch (LATCH_CLK) controlan la transferencia de datos seriales hacia las salidas paralelas de manera sincronizada. Al utilizar VHDL, noté la flexibilidad de este lenguaje para describir y simular el comportamiento lógico del sistema antes de implementarlo físicamente. Durante la configuración, enfrenté desafíos relacionados con la asignación de pines y la correspondencia entre las salidas de la GAL y las entradas de los registros 74HC595, pero estas dificultades me ayudaron a reforzar conceptos sobre la integración entre dispositivos programables y periféricos digitales.

Vázquez Blancas Cesar Said

La combinación de registros de desplazamiento 74HC595 con la GAL22V10, utilizando VHDL como lenguaje de descripción, simplificó la implementación del contador de 0 a 9. En este proceso, aprendí a escribir módulos en VHDL para generar las señales de control necesarias, como el pulso de reloj y las salidas sincronizadas hacia los displays. A pesar de que inicialmente encontré complejo entender la estructura jerárquica de los módulos en VHDL, logré programar un diseño eficiente que traduce el conteo en un formato legible por los displays de 7 segmentos. También experimenté cómo las conexiones en cascada de los registros pueden ampliar la capacidad del sistema para futuros diseños más avanzados, como contadores de mayor escala. Este ejercicio resaltó la importancia de simular el diseño en software antes de implementarlo físicamente, reduciendo errores y optimizando el tiempo de desarrollo.

Salauz Hernández Nerick Francisco

El diseño del sistema para realizar un conteo de 0 a 9 utilizando registros de desplazamiento y una GAL22V10 programada en VHDL fue una experiencia reveladora que integró conceptos teóricos y prácticos de diseño digital. La utilización de VHDL permitió describir de manera estructurada el comportamiento lógico del contador, generando señales precisas para los registros 74HC595. Durante la práctica, enfrenté problemas con las temporizaciones, ya que frecuencias de reloj incorrectas afectaban la sincronización entre la GAL y los registros, lo que causaba errores en la visualización de los números. Sin embargo, ajustar estas señales en el código de VHDL me ayudó a entender la relación crítica entre el hardware y el software en un diseño digital. Este ejercicio también me mostró cómo un diseño modular y reutilizable en VHDL facilita la expansión del sistema a aplicaciones más complejas, como contadores de varias cifras o temporizadores.

5. BIBLIOGRAFIA

- Texas Instruments. (2000). *74HC595 Datasheet: 8-Bit Serial-In, Serial or Parallel-Out Shift Registers with Output Latches*.
- Wakerly, J. (2017). *Digital Design: Principles and Practices* (5.^a ed.). Pearson.
- Roth, C., & John, L. (2014). *Digital Systems Design Using VHDL*. Cengage Learning.
- Tocci, R., Widmer, N., & Moss, G. (2010). *Sistemas digitales: principios y aplicaciones* (11.^a ed.). Pearson.
- Pedroni, V. (2004). *Circuit Design with VHDL*. MIT Press.